

JavaScript,

Where Logic Goes to Die



Why does

```
console.log(999999999999999999999999999)
```

print

10000000000000000

JS numbers lose precision beyond $2^{53} - 1$ (IEEE-754 rounding).



1

Why does `console.log(1 < 2 < 3)`
return `true` ,
but `console.log(3 > 2 > 1)`
return `false`

???

Comparisons run left to right:

$1 < 2 \rightarrow \text{true} \rightarrow \text{true} < 3 \rightarrow 1 < 3 \rightarrow \text{true}$

$3 > 2 \rightarrow \text{true} \rightarrow \text{true} > 1 \rightarrow 1 > 1 \rightarrow \text{false}$



2

Why does `console.log([] == false)`
return `true` ,
but `console.log([] === false)` return
`false`



`[]` is truthy, but `==` forces type coercion while `===` does not this is why `[] == false` is true but `[] === false` is false.



Why does `console.log('5' - 3)`
return `2`,
but `console.log('5' + 3)`
return `3`



- forces number conversion, + prefers
string concatenation.



Why does

`console.log(!!"false" == !!"true")` return
true



!! converts values to boolean, and
since both "false" and "true" are non-
empty strings, they are both truthy
resulting in true == true.



Why does `console.log([] == ![])` return `true`



`[]` is truthy, so `![]` is false — but `==` uses type coercion, converting `[]` to `""` and `false` to `0`, making `"" == 0` $\rightarrow$ `true`.



Why does
`console.log(0.1 + 0.2 === 0.3)`
return `false`



JavaScript uses IEEE-754 floating-point math

- `0.1 + 0.2` becomes
`0.30000000000000004`
- Not exactly `0.3`

Precision issue, not a JS bug



Why does `console.log({ } + [ ]);`
`return "[object Object]"`



When using `+`, JavaScript tries to convert operands to strings.

- `{ }` → `"[object Object]"`
- `[ ]` → `""` (empty string)



Why does `console.log(typeof NaN)` return "number"



NaN is a special numeric value meaning invalid number.



Why does

```
console.log('b' + 'a' + + 'a' + 'a')  
print "banana"
```



- 'b' + 'a' → "ba"
- + 'a' → JavaScript tries to convert 'a' to a number → NaN (Unary + tries to convert a value to a number)
- "ba" + NaN → "baNaN"
- "baNaN" + 'a' → "baNaNNa"

When printed, "baNaNNa" visually looks like "banana".

